Figure 16.3 Model binding involves mapping values from binding sources, which correspond to different parts of a request.

NOTE In Razor Pages, different properties of a complex model can be model-bound to different sources. This differs from minimal APIs, where the whole object would be bound from a single source, and "partial" binding is not possible. Razor Pages also bind to form bodies by default, while minimal APIs cannot. These differences are partly for historical reasons and partly because minimal APIs opts for performance over convenience in this respect.

PageModel properties or page handler parameters?

There are three ways to use model binding in Razor Pages:

- Decorate properties on your `PageModel` with the `[BindProperty]` attribute.
- Add parameters to your page handler method.
- Decorate the whole `PageModel` with `[BindProperties]`.

Which of these approaches should you choose?

The answer to this question is largely a matter of taste. Setting properties on the `PageModel` and marking them with `[BindProperty]` is the approach you'll see most often in examples. If you use this approach, you'll be able to access the binding model when the view is rendered, as you'll see in chapters 17 and 18.

The second approach, adding parameters to page handler methods, provides more separation between the different MVC stages, because you won't be able to access the parameters outside the page handler. On the downside, if you *do* need to display those values in the Razor view, you'll have to copy the parameters across manually to properties that can be accessed in the view.

I avoid the final approach, decorating the `PageModel` itself with `[BindProperties]`. With this approach, *every* property on your `PageModel` takes part in model binding. I don't like the indirection this gives and the risk of accidentally binding properties I didn't want to be model-bound.